

Dataproc + BigQuery Connector Patterns

A Practical Handbook for Reading & Writing BigQuery from Spark

Covers: spark-bigquery-connector • Direct vs indirect writes • Read patterns & predicate pushdown • Partitioning/clustering pass-through • A full hands-on demo you can run yourself (gcloud CLI, PySpark, and a companion notebook)

Prepared for GCP Data Engineering Training

Reflects Google Cloud naming as of July 2026 — Managed Service for Apache Spark

Table of Contents

1. What Is the Spark-BigQuery Connector?
2. Core Concepts & Glossary — every term explained
3. Read Patterns: How Spark Pulls Data From BigQuery
4. Write Patterns: Direct vs. Indirect, Explained
5. Partitioning, Clustering & Schema Pass-Through
6. IAM Roles You Need to Know
7. Hands-On Demo: Step-by-Step
8. Decision Guide: Which Pattern Should You Use?
9. Troubleshooting Common Issues
10. Best Practices Checklist
11. Companion Notebook & Cleanup
12. Quick Reference & Further Reading

1. What Is the Spark-BigQuery Connector?

IMPORTANT: As with the earlier handbooks: Dataproc (clusters) and Dataproc Serverless are now marketed together as the **Managed Service for Apache Spark**. This handbook keeps using 'Dataproc' for familiarity — the connector, its options, and every command below work identically under either name.

The **spark-bigquery-connector** is an open-source library (maintained by Google, in the `GoogleCloudDataproc` GitHub org) that lets Apache Spark treat BigQuery tables as a native data source — you read a BigQuery table straight into a Spark `DataFrame` with `spark.read.format("bigquery")`, and write a `DataFrame` straight into BigQuery with `df.write.format("bigquery")`, with no manual export/import step in between.

It matters because Spark and BigQuery solve different halves of the same problem. BigQuery is excellent at large-scale, ad-hoc, and scheduled SQL analytics but is not a general-purpose programming environment. Spark is excellent at custom transformation logic, ML feature engineering, iterative algorithms, and joining with non-BigQuery sources (files, Kafka, other databases), but isn't a data warehouse. The connector is the bridge: pull data out of BigQuery for Spark-side processing that SQL can't easily express, then push the result back in for everyone else to query.

Where it fits versus the earlier handbooks

If you completed the *Dataproc Serverless & Apache Iceberg* handbook, note the distinction: that handbook covered Spark writing **Iceberg tables on Cloud Storage**, queried by BigQuery via a shared catalog (BigLake metastore) — one physical copy of data, no connector needed. This handbook covers Spark reading and writing **native BigQuery tables** directly, via the connector, which does involve moving data between BigQuery's managed storage and Spark. They're complementary patterns for different situations — Section 8 helps you choose between them.

2. Core Concepts & Glossary

Term	Plain-English Definition
spark-bigquery-connector	The open-source Spark data source connector for BigQuery. Pre-installed on Dataproc cluster image 2.1+ and on all Dataproc Serverless batches — usually no manual JAR installation needed.
BigQuery Storage Read API	The API the connector uses under the hood when reading a BigQuery table into Spark. It streams data directly from BigQuery's storage layer in parallel to Spark executors, in a columnar format, avoiding the old slow path of exporting to Cloud Storage first.
BigQuery Storage Write API	The API used by the connector's DIRECT write method. Writes rows straight into BigQuery's storage layer without an intermediate export/load step.
Direct write method	<code>writeMethod="direct"</code> — Spark writes straight to BigQuery via the Storage Write API. Faster and more immediate, but incurs Storage Write API charges and (in older connector versions) had more edge-case limitations around streaming/appends.
Indirect write method	<code>writeMethod="indirect"</code> (the default) — Spark stages output as Avro/Parquet files in a temporary Cloud Storage bucket, then triggers a standard BigQuery load job to move it into the target table. Cheaper (load jobs are free) but has an extra staging step and isn't instantaneous.
temporaryGcsBucket	The Cloud Storage bucket the connector uses to stage data during an INDIRECT write. Required unless already configured at the cluster/session level; not used at all by DIRECT writes.
persistentGcsBucket / persistentGcsPath	Optional settings that keep the staged files after an indirect write instead of deleting them — useful for debugging or reuse, but adds storage cost if left on indefinitely.
Predicate pushdown	When Spark's <code>.filter()/.where()</code> clauses are translated into a filter that the BigQuery Storage Read API applies server-side, so unnecessary rows are never even transferred to Spark. Reduces both cost and read time.
Column pruning	When Spark's <code>.select()</code> only requests specific columns, the connector asks the Storage Read API for just those columns rather than the whole table — crucial for wide tables.
materializationDataset / materializationProject	When you read from a BigQuery <i>*view*</i> or a raw SQL query (rather than a table) via the connector, BigQuery must first materialize the results into a temporary table before Spark can stream them; this setting controls where that temporary table is created.
SaveMode	Spark's standard write-behavior flag: Append, Overwrite, ErrorIfExists, or Ignore — works the same way with the BigQuery connector as with any other Spark data source.

Term	Plain-English Definition
allowFieldAddition / allowFieldRelaxation	Write-time options controlling whether the connector is allowed to add new columns or relax a column from REQUIRED to NULLABLE on the destination table automatically, similar to BigQuery load job schema update options.
Connector JAR / version pinning	The connector ships as a versioned JAR (e.g. spark-bigquery-with-dependencies_2.12-0.36.1.jar). Dataproc pre-installs a default version per image; you can override it via the dataproc.sparkBqConnector.version batch property or the --jars flag.
BigQuery Storage API session	A short-lived read session created behind the scenes when Spark reads a BigQuery table, used to track and parallelize the streaming read across executors.
Query pushdown (deprecated)	An older connector feature that tried to push some Spark SQL operations down into BigQuery as a query. Removed in recent connector versions in favor of simpler, more predictable read/write semantics — worth knowing if you see it referenced in older tutorials.

3. Read Patterns: How Spark Pulls Data From BigQuery

There are three common ways to read from BigQuery via the connector, in increasing order of flexibility (and decreasing order of efficiency).

Pattern	How	Tradeoff
Read a table directly	<code>.option("table", "project.dataset.table")</code>	Fastest — streams straight via the Storage Read API with full predicate/column pushdown. Use this whenever possible.
Read a view	<code>.option("table", "project.dataset.my_view") + viewsEnabled=true</code>	BigQuery must first materialize the view into a temporary table (billed as a query), then Spark streams from that. Slower and adds query cost; set <code>materializationDataset</code> to control where the temp table lands.
Read an arbitrary SQL query	<code>.option("query", "SELECT ... FROM ...")</code>	Most flexible, but same materialization cost/step as reading a view. Good for pre-aggregating or pre-filtering server-side before Spark even sees the data, when the filter is too complex for simple pushdown.

Example: direct table read with pushdown

```
df = spark.read.format("bigquery") \
    .option("table", "himanshugcpproject.hr_demo.employees") \
    .load()

# This filter and column selection are pushed down to BigQuery server-side --
# only matching rows/columns are ever transferred to Spark.
engineering = df.filter(df.department == "Engineering").select("employee_id", "full_name")
```

NOTE: Predicate pushdown only works for filters the connector can translate — simple equality/range comparisons on top-level columns. Complex UDFs or nested-field filters may not push down, silently falling back to 'read everything, then filter in Spark' — worth checking with `df.explain()` if a read seems slower than expected.

4. Write Patterns: Direct vs. Indirect, Explained

This is the single most consequential setting when writing from Spark to BigQuery — it affects cost, latency, and even which features (like policy tags) are preserved.

Aspect	DIRECT	INDIRECT
Underlying mechanism	BigQuery Storage Write API (writes rows directly)	Stages Avro/Parquet in GCS, then a standard BigQuery load job moves it in
Cost	Storage Write API pricing applies	Free (BigQuery load jobs don't accrue charges); you pay only for the temporary GCS storage
Speed / latency	Faster, more immediate — no staging step	Slower — extra write-to-GCS and load-job steps
Best for	Smaller, latency-sensitive writes; streaming-style appends	Large batch writes where cost matters more than a few extra minutes
Default in	Must be set explicitly	The default for both Dataproc clusters and Managed Service for Apache Spark batches if you don't specify <code>writeMethod</code>
Historical caveat	Older connector versions had a known issue (fixed) where indirect writes could drop column-level policy tags on the destination table	Same caveat applied here before the fix — verify policy tags after any write if you're on an older pinned connector version

Example: indirect write (the default, explicit for clarity)

```
processed.write.format("bigquery") \
  .option("table", "himanshucppproject.hr_demo.employees_processed") \
  .option("writeMethod", "indirect") \
  .option("temporaryGcsBucket", "himanshucppproject-spark-temp") \
  .mode("overwrite") \
  .save()
```

Example: direct write

```
processed.write.format("bigquery") \
  .option("table", "himanshucppproject.hr_demo.employees_processed") \
  .option("writeMethod", "direct") \
  .mode("append") \
  .save()
```

Note: no temporaryGcsBucket needed for direct writes.

IMPORTANT: One notable exception: if you're using a **BigQuery Studio notebook** with a Spark Connect session (Section 6 territory in the BigQuery docs), the connector is **pre-configured for DIRECT writes by default** — the opposite default from standalone Dataproc batches. Always check which environment you're in rather than assuming the default.

5. Partitioning, Clustering & Schema Pass-Through

When Spark creates a new BigQuery table (rather than writing into an existing one), you can specify BigQuery-native partitioning and clustering directly through connector options — the resulting table is a normal, fully-featured partitioned/clustering BigQuery table, not a workaround.

```
processed.write.format("bigquery") \
  .option("table", "himanshucppproject.hr_demo.events_partitioned") \
  .option("partitionField", "event_date") \
  .option("partitionType", "DAY") \
  .option("clusteredFields", "user_id,event_type") \
  .option("writeMethod", "direct") \
  .mode("overwrite") \
  .save()
```

Schema types generally map cleanly between Spark and BigQuery (StringType↔STRING, LongType↔INT64, DoubleType↔FLOAT64, StructType↔RECORD, ArrayType↔REPEATED, and so on). Two things worth knowing explicitly:

- Nullability is preserved by default; use `allowFieldRelaxation=true` if you need the connector to relax a REQUIRED destination column to NULLABLE automatically on write.
- New columns present in the Spark DataFrame but not in the destination table are rejected unless `allowFieldAddition=true` is set.

6. IAM Roles You Need to Know

Role (display name)	Role ID	What it lets you do
BigQuery Data Editor	roles/bigquery.dataEditor	Needed on the Dataproc/Spark job's service account to write results into BigQuery tables, and to read via views/queries which require creating temporary tables.
BigQuery Data Viewer	roles/bigquery.dataViewer	Minimum needed to read an existing BigQuery table into Spark.
BigQuery Job User	roles/bigquery.jobUser	Needed to run the load jobs (indirect writes) or query jobs (materializing views/queries) that the connector triggers on your behalf.
BigQuery Read Session User	roles/bigquery.readSessionUser	Specifically needed to create BigQuery Storage Read API sessions — required for reads even if you already have Data Viewer.
Storage Object Admin	roles/storage.objectAdmin	Needed on the temporary GCS staging bucket for INDIRECT writes (not required at all for DIRECT writes).

IMPORTANT: roles/bigquery.readSessionUser is the one people most often forget — Data Viewer alone is not enough to read via the Storage Read API. If reads fail with a permission error even though the service account can clearly see the table in the console, this is the first thing to check.

7. Hands-On Demo — Step by Step

This demo submits a Dataproc Serverless batch that reads the `hr_demo.employees` table (from earlier handbooks) into a Spark DataFrame, applies a transformation, and writes the result to a new BigQuery table — once using `INDIRECT` and once using `DIRECT`, so you can compare them side by side.

NOTE: Time required: ~20–30 minutes.

Prerequisites

- The `hr_demo.employees` table from the Data Catalog handbook (or any BigQuery table — adjust the table name in the script).
- A Cloud Storage bucket for `INDIRECT`-write staging.
- Dataproc Editor on your account; BigQuery Data Editor + Job User + Read Session User, and Storage Object Admin (for the staging bucket) on the batch job's service account.

Set shell variables

```
export PROJECT_ID="himanshugcpproject"
export REGION="us-central1"
export TEMP_BUCKET="${PROJECT_ID}-spark-temp"

gcloud config set project $PROJECT_ID
gcloud storage buckets create gs://${TEMP_BUCKET} --location=${REGION}
```

STEP 1 — Grant the batch job's service account the roles it needs

```
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
export SERVICE_ACCOUNT="${PROJECT_NUMBER}[email protected]"

for ROLE in roles/bigquery.dataEditor roles/bigquery.jobUser \
            roles/bigquery.readSessionUser roles/storage.objectAdmin; do
  gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member="serviceAccount:${SERVICE_ACCOUNT}" \
    --role="$ROLE"
done
```

STEP 2 — Write the PySpark job (read + transform + write both ways)

Save as `connector_demo.py`:

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, upper

spark = SparkSession.builder.appName("ConnectorPatternsDemo").getOrCreate()

# --- READ: direct table read with pushdown ---
df = spark.read.format("bigquery") \
    .option("table", "PROJECT_ID.hr_demo.employees") \
    .load()

transformed = df.filter(col("department") == "Engineering") \
    .withColumn("department_upper", upper(col("department")))

transformed.show()

# --- WRITE (INDIRECT): staged via GCS, loaded via BQ load job ---
transformed.write.format("bigquery") \
    .option("table", "PROJECT_ID.hr_demo.employees_indirect") \
    .option("writeMethod", "indirect") \
    .option("temporaryGcsBucket", "TEMP_BUCKET") \
    .mode("overwrite") \
    .save()

# --- WRITE (DIRECT): straight via Storage Write API ---
transformed.write.format("bigquery") \
    .option("table", "PROJECT_ID.hr_demo.employees_direct") \
    .option("writeMethod", "direct") \
    .mode("overwrite") \
    .save()

print("Done. Compare hr_demo.employees_indirect vs hr_demo.employees_direct in BigQuery.")

```

Replace `PROJECT_ID` and `TEMP_BUCKET` placeholders with your actual values before submitting.

STEP 3 — Submit the batch job

```

gcloud dataproc batches submit pyspark connector_demo.py \
  --project=${PROJECT_ID} \
  --region=${REGION} \
  --deps-bucket=${TEMP_BUCKET} \
  --version=2.2

```

The connector is pre-installed on Dataproc Serverless — no `--jars` flag needed unless you want a specific non-default connector version (see Section 10).

STEP 4 — Compare the two output tables in BigQuery

```

SELECT table_name, creation_time, row_count, size_bytes
FROM `PROJECT_ID.hr_demo.__TABLES__`
WHERE table_name IN ('employees_indirect', 'employees_direct');

-- Also check the Jobs Explorer in BigQuery: the indirect write shows up
-- as a visible LOAD job; the direct write does not (no load job is created).

```

8. Decision Guide: Which Pattern Should You Use?

Scenario	Recommended pattern	Why
Large nightly batch ETL, cost-sensitive	INDIRECT	Load jobs are free; the extra few minutes of staging rarely matters for a scheduled batch.
Near-real-time or streaming-style appends	DIRECT	No staging delay; better fit for low-latency write patterns.
Writing to a table with policy tags, on an older pinned connector version	DIRECT (or upgrade connector)	Older indirect-write versions had a bug dropping policy tags; either use direct or upgrade past the fix.
Reading a huge table, need only a few columns/rows	Direct table read with filter+select pushdown	Predicate/column pushdown minimizes data transferred — by far the biggest read-side cost/speed lever.
Reading from a BigQuery view or complex SQL	query/view read pattern, tune materializationDataset	Accept the one-time materialization cost; place the temp dataset somewhere with appropriate retention/cost controls.
Writing a brand-new table that needs partitioning/clustering	Either method + partitionField/clusteringField options	Both write methods support creating partitioned/clustering tables; choose based on the cost/latency tradeoff above.

9. Troubleshooting Common Issues

Symptom	Likely Cause / Fix
Read fails with a permission error despite having Data Viewer	Missing roles/bigquery.readSessionUser — required specifically for BigQuery Storage Read API sessions, separate from ordinary table read permission.
Indirect write fails: “temporaryGcsBucket not set”	INDIRECT requires an explicit temporaryGcsBucket option (or a pre-configured fs.gs.system.bucket on the cluster) — DIRECT does not need this at all.
Write succeeds but policy tags are missing on the destination table	Known historical bug in older indirect-write connector versions. Upgrade the connector version or switch to DIRECT for that specific write.
Reads are much slower than expected on a filtered query	The filter likely isn't being pushed down (e.g. it's on a nested/repeated field, or uses a UDF). Check with <code>df.explain()</code> and consider restructuring the filter to a simple top-level comparison.
“Access Denied: BigQuery: Permission denied while getting Drive credentials” on read	Usually unrelated to your own data — happens if the source table is itself a BigQuery table backed by a Google Sheet/Drive file the job's service account can't access.
Built-in connector version conflicts with a version you specified via <code>--jars</code>	On Dataproc image 2.1+ / Serverless, the built-in connector takes precedence over <code>--jars</code> . To use a different version, set the <code>dataproc.sparkBqConnector.version</code> batch property instead.

10. Best Practices Checklist

- Default to INDIRECT for large batch writes (it's free) and reserve DIRECT for latency-sensitive or streaming-style writes where the Storage Write API cost is justified.
- Always push filters and column selection down where possible — `.filter()` and `.select()` before any wide transformation, not after — to minimize data actually transferred from BigQuery.
- Pin the connector version explicitly for production jobs (`dataproc.sparkBqConnector.version`) rather than relying on whatever the current image's default happens to be.
- Grant roles/`bigquery.readSessionUser` alongside Data Viewer as a standard pairing — don't wait to discover it's missing at demo time.
- When reading from views or ad-hoc queries, set `materializationDataset` explicitly to a short-retention dataset you control, rather than letting temporary tables land wherever BigQuery defaults to.
- For new tables, specify `partitionField/clusteredFields` on write rather than creating an unpartitioned table and fixing it later — partitioning after the fact requires a full rewrite.
- After any indirect write to a policy-tagged table, spot-check that the tags survived, especially on older pinned connector versions.

11. Companion Notebook & Cleanup

A companion notebook (`dataproc_bigquery_connector_demo.ipynb`) accompanies this handbook. Like the Iceberg demo notebook, it submits and monitors a Dataproc Serverless batch (since the connector runs inside the Spark job, not the notebook kernel), then pulls back both output tables via the BigQuery client library so you can compare row counts, schemas, and job-history metadata for the DIRECT vs INDIRECT writes side by side.

Cleanup

```
bq rm -f -t ${PROJECT_ID}:hr_demo.employees_indirect
bq rm -f -t ${PROJECT_ID}:hr_demo.employees_direct
gcloud storage rm --recursive gs://${TEMP_BUCKET}
gcloud storage buckets delete gs://${TEMP_BUCKET}
```

12. Quick Reference & Further Reading

One-line summary of the whole workflow

```
spark.read.format("bigquery").option("table", ...).load() -- streams via Storage Read API,  
with filter/column pushdown where possible.  
  
df.write.format("bigquery").option("table", ...).option("writeMethod", DIRECT|INDIRECT).save()  
-- DIRECT: straight via Storage Write API (costs more, faster).  
-- INDIRECT (default): stage to GCS, then a free BigQuery load job (cheaper, slower).
```

Official documentation to bookmark

- spark-bigquery-connector (GitHub, options reference) — github.com/GoogleCloudDataproc/spark-bigquery-connector
- Use the Spark BigQuery connector — docs.cloud.google.com/dataproc/docs/tutorials/bigquery-connector-spark-example
- BigQuery Storage Read/Write API overview — docs.cloud.google.com/bigquery/docs/reference/storage
- Run PySpark code in BigQuery Studio notebooks — docs.cloud.google.com/bigquery/docs/use-spark

End of handbook. Pair this with the companion notebook for the live demo, and the Troubleshooting table in Section 9 during Q&A.